

Reinforcement Learning (RL) "Good puppy ..." learning.

Feedback is not provided for every step, but only for final result.

Examples – learning to win in games (Atari, Alpha Go), robotics, etc.

Policy Gradients:

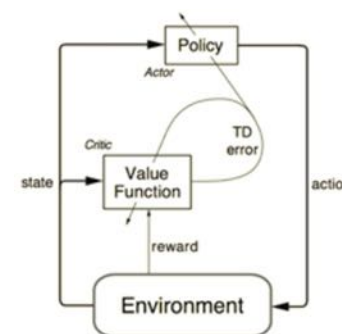
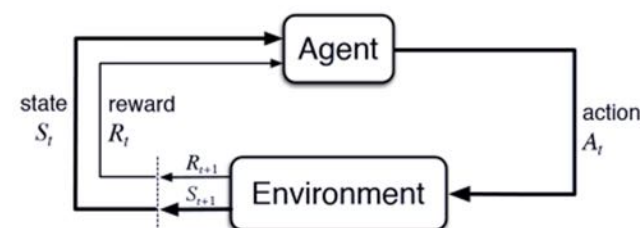
Learn policy map. A **software agent** selects **actions** using a map called "**policy**". The **policy map** gives the probability of taking **action** "a" when in a certain **state** "s".

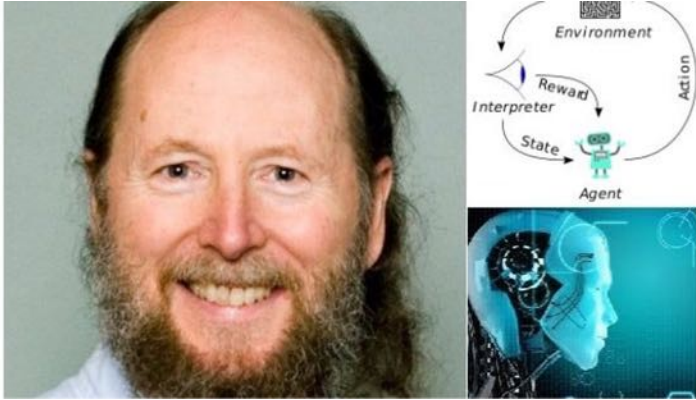
Deep Q-Learning: (Watkins 1989, DeepMind 2014)

Learn Q-function to maximize the expected value of the total reward over all successive steps. The **value function** estimates how good it is to be in a given **state**. The goal is to find a **policy** with **maximum expected cumulative return**.

Actor-Critic Architecture: the **actor** - tunes the parameter θ for the **policy function** (decide the best action for a specific state). The **critic** is used for evaluating the **policy function** estimated by the **actor** according to the temporal difference (TD) error (TD = difference between estimated reward and actual reward received).

The **exploration vs. exploitation trade-off**. Multi-armed bandit problem in finite MDPs (Markov Decision Processes). One-armed bandit - a slot machine with some probability distribution of the reward. Multi-armed bandit - several slot machines available. So you have fixed limited set of resources (your moves), and you need to allocate this resource between competing (alternative) choices (slot-machines) to increase reward





Rich Sutton, the Father of Reinforcement Learning (RL), Temporal Difference (TD) learning and policy gradient methods.

PhD (1984) in Computer science "Temporal Credit Assignment in Reinforcement Learning", where he introduced **actor-critic** architectures and "temporal credit assignment".

Since 2003 - in Alberta, Canada.

Since 2017 - Google DeepMind in Alberta, Canada

Book (1998) : Reinforcement Learning. An Introduction - By Richard S. Sutton and Andrew G. Barto:

<http://www.incompleteideas.net/> -

Interview: <https://www.kdnuggets.com/2017/12/interview-rich-sutton-reinforcement-learning.html>

YouTube: <https://www.youtube.com/watch?v=QqLcniN2VAk> -

"Reinforcement learning is learning from rewards, by trial and error, during normal interaction with the world. This makes it very much like natural learning processes and unlike supervised learning (in which learning only happens during the training phase).

"Deep reinforcement learning is the combination of deep learning and reinforcement learning.

"... reinforcement learning is the future of AI. Reinforcement learning is the best representative of the idea that an intelligent system must be able to learn on its own, without constant supervision.

"... the key is to learn from ordinary unsupervised data. ... in the near term this will be done by focusing on **"prediction learning"**. **Prediction learning** may shortly become a strong buzzword. "

"... a next big step will come when we have systems that plan with a learned model of the world ... "

Frequently Used Terms in Reinforcement Learning

- **Agent** - takes actions (car, drone, Super Mario, etc.)
- **Actions (A)** - set of all possible moves the agent can make
- **Discount Factor** - future rewards are multiplied by discount factor (on each step) to make future rewards worth less than immediate rewards. We discount rewards, or lower their estimated value, the further into the future they occur. This enforces a "short-term hedonism" in the agent. Keynes: "In the long run, we are all dead." That's why you discount future rewards.
- **Environment** - (includes laws of physics, rules of society, etc.)
- **State (S)** - situation, place & moment, instantaneous configuration, ...
- **Reward (R)** - feedback by which we measure the success or failure of an agent's actions.
- **Policy (π)** - the strategy that the agent employs to determine the next action based on the current state. It maps states to actions (the actions that promise the highest reward).
- **Value (V)** - The expected long-term return with discount, as opposed to the short-term reward R . $V_\pi(s)$ is defined as the expected long-term return of the current state under policy π . **Value function** is used to find a policy that maximizes the return by maintaining a set of estimates of expected returns for some policy (usually either the **"current" [on-policy]** or the **optimal [off-policy]** one).
- **Q-value** or action-value (Q): Q-value is similar to Value, except that it takes an extra parameter, the current action a . $Q_\pi(s, a)$ refers to the long-term return of the current state s , taking action a under policy π . Q maps state-action pairs to rewards. Note the difference between Q and policy.
- **Trajectory** - A sequence of states and actions that influence those states. From the Latin "to throw across." The life of an agent is but a ball tossed high and arching through space-time.

from Wikipedia, November 2018)

Comparison of reinforcement learning algorithms [\[edit \]](#)

Algorithm	Description	Model	Policy	Action Space	State Space	Operator
Monte Carlo	Every visit to Monte Carlo	Model-Free	Off-policy	Discrete	Discrete	Sample-means
Q-learning	State–action–reward–state	Model-Free	Off-policy	Discrete	Discrete	Q-value
SARSA	State–action–reward–state–action	Model-Free	On-policy	Discrete	Discrete	Q-value
Q-learning - Lambda	State–action–reward–state with eligibility traces	Model-Free	Off-policy	Discrete	Discrete	Q-value
SARSA - Lambda	State–action–reward–state–action with eligibility traces	Model-Free	On-policy	Discrete	Discrete	Q-value
DQN	Deep Q Network	Model-Free	Off-policy	Discrete	Continuous	Q-value
DDPG	Deep Deterministic Policy Gradient	Model-Free	Off-policy	Continuous	Continuous	Q-value
A3C	Asynchronous Actor-Critic Algorithm	Model-Free	Off-policy	Continuous	Continuous	Q-value
NAF	Q-Learning with Normalized Advantage Functions	Model-Free	Off-policy	Continuous	Continuous	Advantage
TRPO	Trust Region Policy Optimization	Model-Free	On-policy	Continuous	Continuous	Advantage
PPO	Proximal Policy Optimization	Model-Free	On-policy	Continuous	Continuous	Advantage

Model-based vs Model-free

Model is a simulation of dynamics of environment

model-based algorithms become impractical as the state space and action space grows ($S \times S \times A$, for a tabular setup)

model-free algorithms rely on trial-and-error to update its knowledge. Don't require space to store all states & actions

On-policy v.s. Off-policy

An off-policy agent learns the value based on the action **"a"** obtained from **another policy**. (Greedy policy in Q-learning).

An on-policy agent learns the value based on its current action **"a"** derived from the **current policy**,

On Policy vs Off Policy

- Q, an off-policy learner:

$$Q(a, s) \leftarrow Q(a, s) + \alpha \cdot (r_s + \gamma \max_{a'} Q(a', s') - Q(a, s))$$

- SARSA, an on-policy learner:

$$Q(a, s) \leftarrow Q(a, s) + \alpha \cdot (r_s + \gamma \cdot Q(a', s') - Q(a, s))$$

Q-Learning

Q-Learning is an off-policy, model-free RL algorithm based on the well known Bellman Equation:

$$v(s) = \mathbb{E}[R_{t+1} + \lambda v(S_{t+1}) | S_t = s]$$

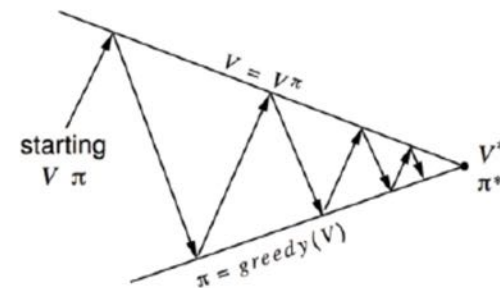
E in the above equation refers to the expectation, while λ refers to the discount factor.

We can rewrite it in the form of Q-value:

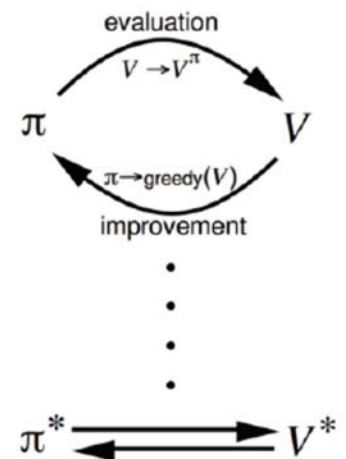
$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}[r_{t+1} + \lambda r_{t+2} + \lambda^2 r_{t+3} + \dots | s, a] \\ &= \mathbb{E}_{s'}[r + \lambda Q^\pi(s', a') | s, a] \end{aligned}$$

The goal is to maximize the Q-value.

Policy iteration (in Q-learning)



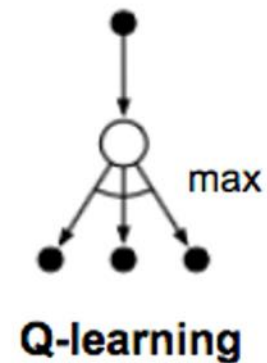
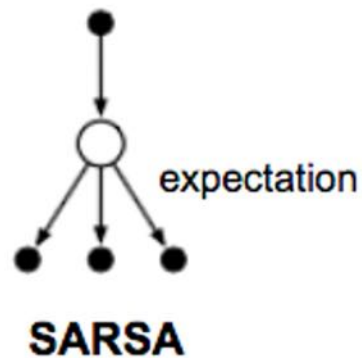
Policy evaluation Estimate v_π
 Any policy evaluation algorithm
 Policy improvement Generate $\pi' \geq \pi$
 Any policy improvement algorithm



SARSA = State-Action-Reward-State-Action

SARSA is similar to Q-learning, but SARSA is an **on-policy** algorithm.

This means that SARSA learns the Q-value based on the action performed **by the current policy** instead of the **greedy policy**.



Deep Q Network (DQN)

You can view Q-learning as updating numbers in a 2-dim array [Action, State].
If agent encounters a new state (which is not in the array) - it has no clue which action to take.
In other words, Q-learning agent does not have the ability to estimate value for unseen states.

To deal with this problem, DQN uses a CNN (Convolutional Neural Network) instead of 2-dim array - to estimate the Q-value function.

The input for the network is the current state,
the output is the corresponding Q-value for each of the action.

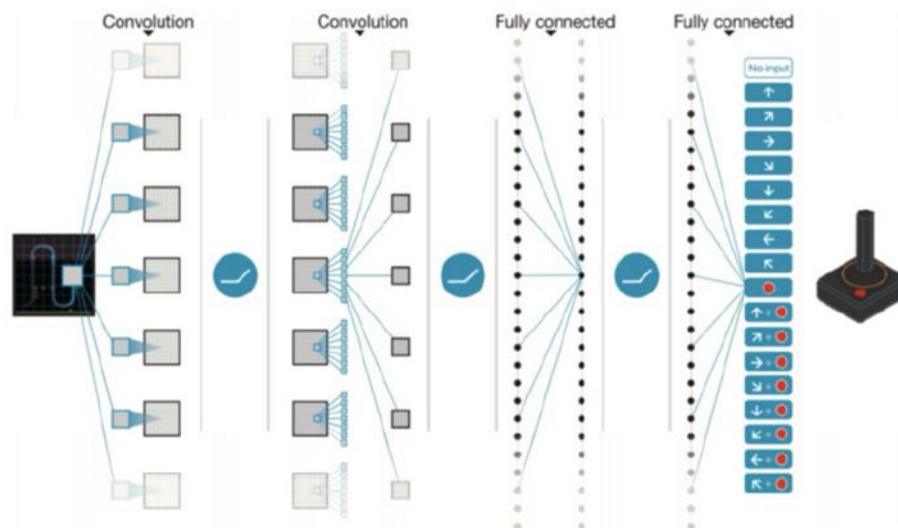
Atari 2013

Experience Replay:

Problem - training samples are highly correlated and not data-efficient. Solution is to store sample transitions - and then replay them randomly.

Separate Target Network:

Problem - fluctuations. Solution - use separate target Q Network (same as NN estimating value) - and reset it to another every "C" steps to reduce fluctuations.



DQN Atari Example (<https://zhuanlan.zhihu.com/p/25239682>)

DDPG = Deep Deterministic Policy Gradient (Actor-Critic architecture).

Problem with DQN - action space is discrete, so it may become too large, difficult to train/converge the model.

DDPG relies on the actor-critic architecture:

- actor is used to tune the parameter(s) for policy function, i.e. decide the best action for a specific state.
- critic is used for evaluating the policy function estimated by the actor according to the temporal difference (TD) error.

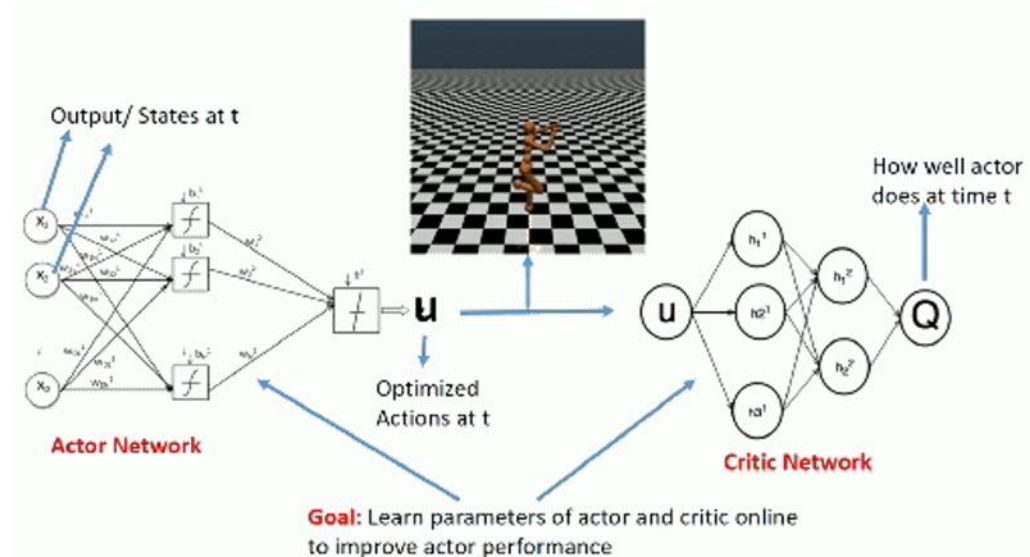
TD learning is a way to learn how to predict a value depending on future values of a given state.

Q-learning is a specific type of TD learning for learning Q-value.

DDPG also borrows DQN the ideas of **experience replay** and **separate target network**.

An issue for DDPG is that it seldom performs exploration for actions. A solution for this is adding noise on the parameter space (or the action space).

One commonly used noise is Ornstein-Uhlenbeck Random Process (a stationary Gauss–Markov process - roughly speaking, describes the velocity of a massive Brownian particle under the influence of friction).



Advantage (A):

$$A(s, a) = Q(s, a) - V(s)$$

We subtract the Value of the state $V(s)$ from $Q(s,a)$ values.
This helps to see which Q-s are better ("increase contrast").

$$A(s, a) = \underbrace{Q(s, a)}_{\substack{\text{q value for action a} \\ \text{in state s}}} - \underbrace{V(s)}_{\substack{\text{average} \\ \text{value} \\ \text{of that} \\ \text{state}}}$$

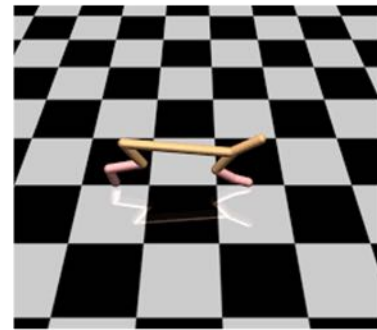
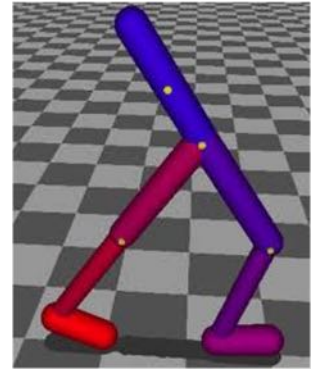
TRPO = Trust Region Policy Optimization

TRPO is an **on-policy continuous** algorithm using **actor-critic** architecture and using "**Advantage**" instead of Q-function. It is similar to DDPG (Deep Deterministic Policy Gradient) , but modifies how the policy parameters of the actor are updated.

The goal is to be able to update parameters in a way that guarantees policy improvement, that is the expected discounted long-term reward η to be always increasing.

TRPO algorithm uses a “surrogate” objective function containing a probability ratio between current policy and the next policy. By choosing the new policy parameters which maximizes the expectation subject to the KL divergence constraint, a lower bound of the expected long-term reward η is guaranteed.

TRPO has achieved great and consistent high performance, but unfortunately the computation and implementation of it is extremely complicated. A Fisher Information Matrix, a second-order derivative of KL divergence, is used to approximate the KL term. This results in computing several second-order matrices, which requires a great amount of computation. In the TRPO paper, Conjugate Gradient (CG) algorithm was used to solve the constrained optimization problem so that the Fisher Information Matrix does not need to be explicitly computed. Yet, CG makes implementation more complicated.



PPO = Proximal Policy Optimization (OpenAI version)

PPO gets rid of the computation created by constrained optimization (KL divergence constraint).

The PPO uses a clipped surrogate objective function which is cheaper to calculate.

The idea of TRPO's constraint is disallowing the policy to change too much. PPO achieves this by slightly modifying TRPO's objective function with a penalty for having a too large policy update.

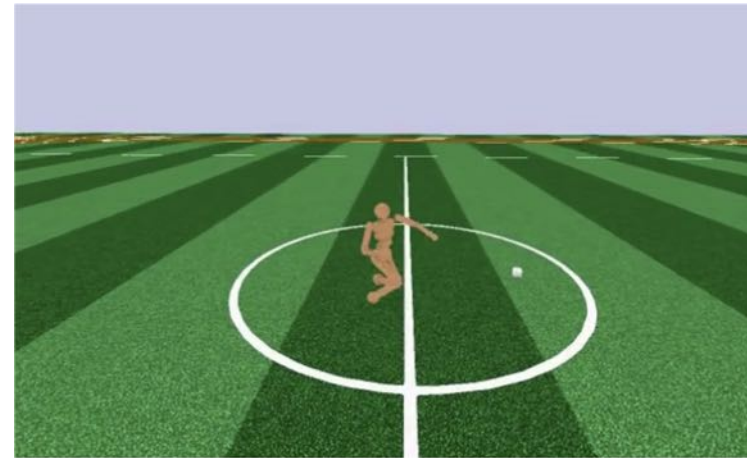
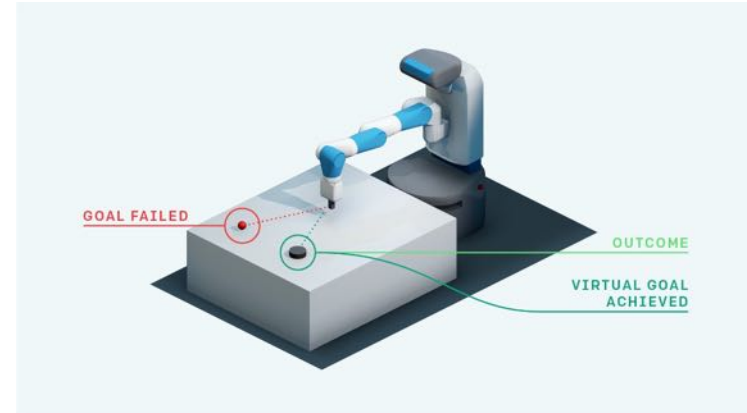
Essentially, it restricts the range that the new policy can vary from the old one.

In practice, loss function error and entropy bonus should also be considered during implementation.

Empirically the PPO's performance is better than TRPO.

In fact, thanks to its lightness and ease of implementation, PPO has become the default RL algorithm of OpenAI

(<https://blog.openai.com/openai-baselines-ppo/>).



Summary of RL algorithms

All the discussed RL algorithms are model-free. That is, none of them are trying to estimate the objective function. Instead, they update their knowledge based on trial-and-error.

SARSA is the only "on-policy", learning value based on its current action.

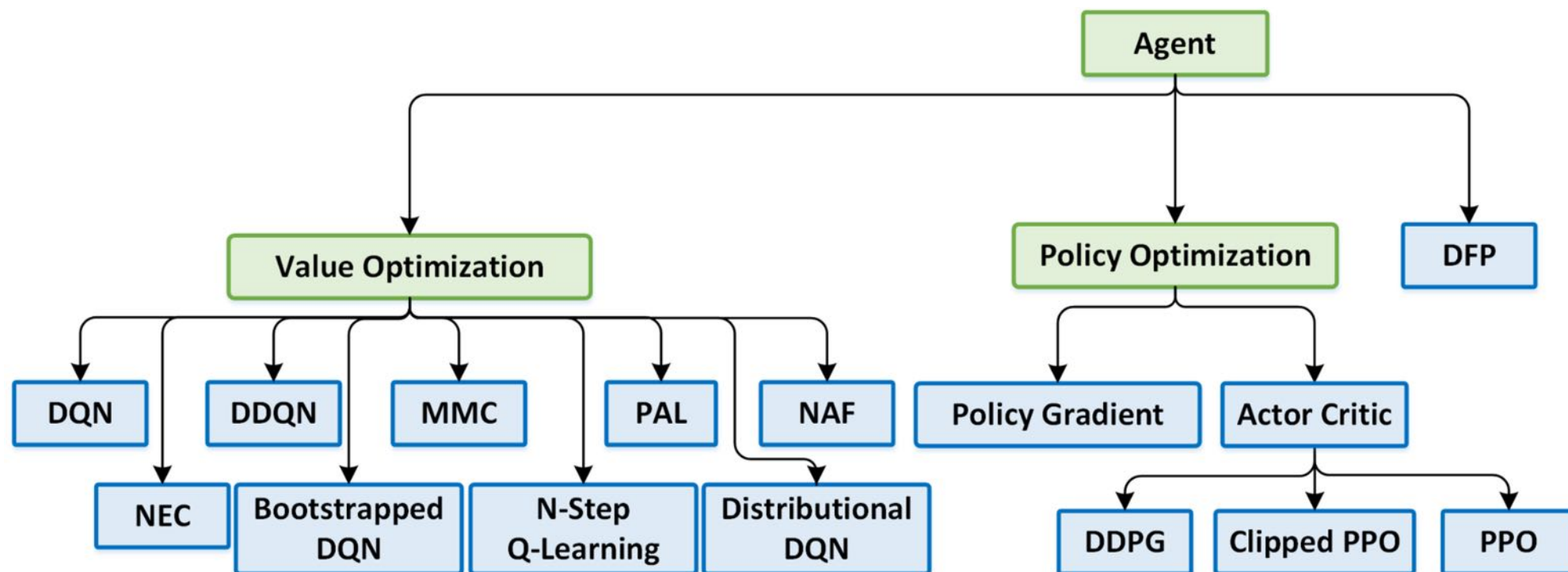
DQN was a huge improvement from a discrete observation space to a continuous one, allowing the agent to handle unseen state.

DDPG was another breakthrough that enables agent to perform continuous actions with policy gradient, broadening the application of RL to more tasks such as control.

TRPO improves the performance of DDPG as it introduces a surrogate objective function and a KL divergence constraint, guaranteeing non-decreasing long-term reward.

PPO further optimizes TRPO by modifying the surrogate objective function, which improves the performance as well as decreasing the complexity of implementation and computation.

Variety of Reinforcement Learning algorithms <https://nervanasystems.github.io/coach/>



A History of Reinforcement Learning - Prof. A.G. Barto, July 2018

- <https://www.youtube.com/watch?v=ul6B2oFPNDM>

Steeve Huang - Introduction to Various Reinforcement Learning Algorithms.

Part I (Q-Learning, SARSA, DQN, DDPG):

- <https://towardsdatascience.com/introduction-to-various-reinforcement-learning-algorithms-i-q-learning-sarsa-dqn-ddpg-72a5e0cb6287>

Part II (TRPO, PPO):

- <https://towardsdatascience.com/introduction-to-various-reinforcement-learning-algorithms-part-ii-trpo-ppo-87f2c5919bb9>

Hindsight Experience Replay (HER) (2017-2018) - a novel technique called Hindsight Experience Replay (HER) which allows sample-efficient learning from rewards which are sparse and binary - and therefore avoid the need for complicated reward engineering (demo on a robotic arm). - <https://arxiv.org/abs/1707.01495>

Technology for Reinforcement Learning:

Google Dopamine (TensorFlow) - <https://github.com/google/dopamine>

Facebook Horizon (PyTorch, Caffe, Spark) - <https://code.fb.com/ml-applications/horizon/>

Intel Reinforcement Learning Coach - <https://github.com/NervanaSystems/coach>